

WEB DEV-II Question Bank

Section A – Short Answer Questions (2 Marks)

1. Define the DOM and explain its role in web development.
2. Explain DOM vs HTML.
3. Difference between `textContent`, `innerHTML`, and `innerText`
4. Difference between `getElementById()`, `getElementsByClassName()`, `querySelector()`, `querySelectorAll()`
5. What are `classList` methods (`add/remove/toggle`)?
6. Discuss microtask and macrotask queue.
7. Difference between `setTimeout()` and `setInterval()`
8. What is Callback Hell? Explain its disadvantages.
9. What are Promise states in JavaScript?
10. Explain the Event Loop and how asynchronous code executes in JavaScript. (*merged with microtask/macrotask concept*)
11. What is JSX in React?
12. What is one-way data binding?
13. What are hooks? State the rules of hooks.
14. Demonstrate the role of `useState` using a simple example.
15. Define props in React with a simple use case.
16. Differentiate between Local Storage and Session Storage.
17. Explain the role of Babel in converting modern JavaScript code.
18. How does React manage frequent UI updates efficiently?
19. Explain the use of dependency array in `useEffect`.
20. Disadvantage of props drilling.

Section B – Long Answer Questions (8 Marks)

1. Discuss event handling in JavaScript using `addEventListener()` with different event types.
2. Explain event propagation. Differentiate between bubbling and capturing. Also discuss the role of event listeners.
3. Create a simple webpage using HTML, CSS, and JavaScript that demonstrates button click events and event removal.
 - Add a **button** on the webpage.
 - Add a **text paragraph/heading** with some default text.
 - When the **button is clicked**:
 - Ø The text should change to a new message.
 - Ø The text color should also change.

After performing the action once, use `removeEventListener()` so that the button click event is removed.
4. Explain execution context in JavaScript including GEC, FEC, and call stack.
5. Explain the lifecycle of a Promise:
 - Problem of Callback Hell
 - Definition of Promise
 - States (Pending, Fulfilled, Rejected)
 - Consumers (`.then`, `.catch`, `.finally`)
 - Implementation example

6. Explain async/await with error handling using try...catch.
7. Write the code-- how to fetch API In React (<https://jsonplaceholder.typicode.com/users>)
Sample JSON :{
 "username": "Bret",
 "email": "Sincere@april.biz".
}
 · Fetch data for all users
 · Render username and email of each user
 Display results
8. Explain component lifecycle methods in React and how they are implemented using useEffect with examples.
9. Create a React component with two buttons and a counter state.
 - First button shows popup every time when clicked
 - Second button updates counter without popup when clicked

Use useState for the counter and useEffect to control when the popup is shown.

10. Design and implement a User Information Form using React with the following requirements:
 - 1) Create a form that includes the following input fields:

Name (Text Input)
Age (Number Input)
Gender (Select Box)
Language (Checkbox)
 - 2) On form submission, display the entered data on the screen.
11. Create a React functional component using useState to implement a counter application(increment, decrement, reset)
12. Explain Context API and compare it with props drilling. Also implement data sharing across nested components.
13. Discuss disadvantages of props drilling and how Context API solves them.
14. Explain routing with example in React and create multiple pages with navigation.

Outcome Based Questions: Guess the output with reason of the output.

```
console.log("Start");
```

```
async function test() {  
  console.log("Inside function");
```

```
  await Promise.resolve();
```

```
  console.log("After await");  
}
```

```
test();
```

```
setTimeout(() => {  
  console.log("Timeout");  
}, 0);
```

```
console.log("End");
```